

4.3、文法的类型

通过对产生式施加不同的限制，Chomsky 将文法分为四种类型：

- **0型文法**：对任一产生式 $\alpha \rightarrow \beta$ ，都有 $\alpha, \beta \in (V_N \cup V_T)^*$ ，且 α 至少含一个**非终极符**

0型文法G生成的语言称为**0型语言**或称**递归可枚举集**。

4.3、文法的类型

- **1型文法：**或称**上下文有关文法**。对任一产生式 $\alpha \rightarrow \beta$ ，都有 $|\beta| \geq |\alpha|$ 。仅 $S \rightarrow \epsilon$ 除外，但 S 不能出现于产生式右部。
- 1型文法相应的语言 $L(G)$ 称为1型语言或**上下文有关语言**。

例： **1**型文法 **G[S]**:

$S \rightarrow CD$

$Ab \rightarrow bA$

$C \rightarrow aCA$

$Ba \rightarrow aB$

$C \rightarrow bCB$

$Bb \rightarrow bB$

$AD \rightarrow aD$

$BD \rightarrow bD$

$Aa \rightarrow bD$

2型文法： 也称**上下文无关文法**(Context Free Grammar, CFG) 。对任一产生式 $\alpha \rightarrow \beta$, 都有 $\alpha \in V_N$, $\beta \in (V_N \cup V_T)^*$ 。

例1: $G[S]$ $S \rightarrow AB$
 $A \rightarrow BS \mid 0$
 $B \rightarrow SA \mid 1$

例:

$S \rightarrow L \mid LT$

$T \rightarrow L \mid D \mid TL \mid TD$

$L \rightarrow a \mid b \mid c \mid d \mid \dots \mid z$

$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9$

例2: $G[E] = (\{ E \} , \{ +, *, i, (,) \} , P , E)$

$P: \{ E \rightarrow i,$
 $E \rightarrow E + E,$
 $E \rightarrow E * E,$
 $E \rightarrow (E) \}$

E表示算术表达式，i表示程序的“变量”

该文法定义了由变量i，+，*，(和)组成的算术表达式的语法结构，即：

- 1) 变量是算术表达式； $E \rightarrow i$**
- 2) 若 E_1 和 E_2 是算术表达式，则 $E_1 + E_2$ ， $E_1 * E_2$ 和 (E_1) 也是算术表达式**

3型文法：任一产生式 $\alpha \rightarrow \beta$ 的形式都为
 $A \rightarrow aB$ 或 $A \rightarrow a$ ，其中 $A, B \in V_N$ ， $a \in V_T^*$ 。

3型文法称为**正规文法**或**右线性文法**，识别系统是有限自动机。

例：G[S]

$S \rightarrow 0A \mid 1B \mid 0$

$A \rightarrow 0A \mid 1B \mid 0S$

$B \rightarrow 1B \mid 1 \mid 0$

G[I] $I \rightarrow iT$

$I \rightarrow i$

$T \rightarrow iT$

$T \rightarrow dT$

$T \rightarrow i$

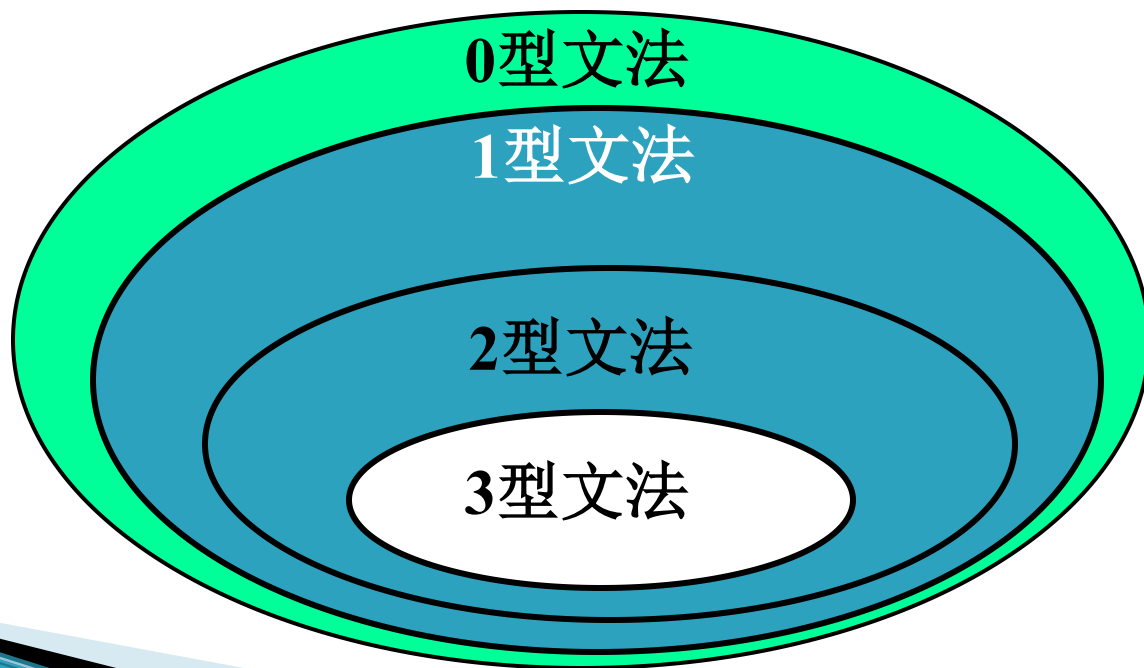
$T \rightarrow d$

左线性文法：任一产生式 $\alpha \rightarrow \beta$ 的形式都为
 $A \rightarrow Ba$ 或 $A \rightarrow a$ ，其中 $A, B \in V_N$ ，
 $a \in V_T^*$

左线性文法和右线性文法都称为**正则文法**。
正则文法能描述程序设计语言的多数单词。

一般称这四类文法为：非限制文法（短语文法）、上下文有关文法、上下文无关文法和正则文法。它们构成的语言类成为**乔姆斯基**层次（Chomsky hierarchy）。这些文法表示计算能力的不同层次。

四种文法之间的**逐级“包含”**关系



四种文法之间的关系

➤ 逐级限制

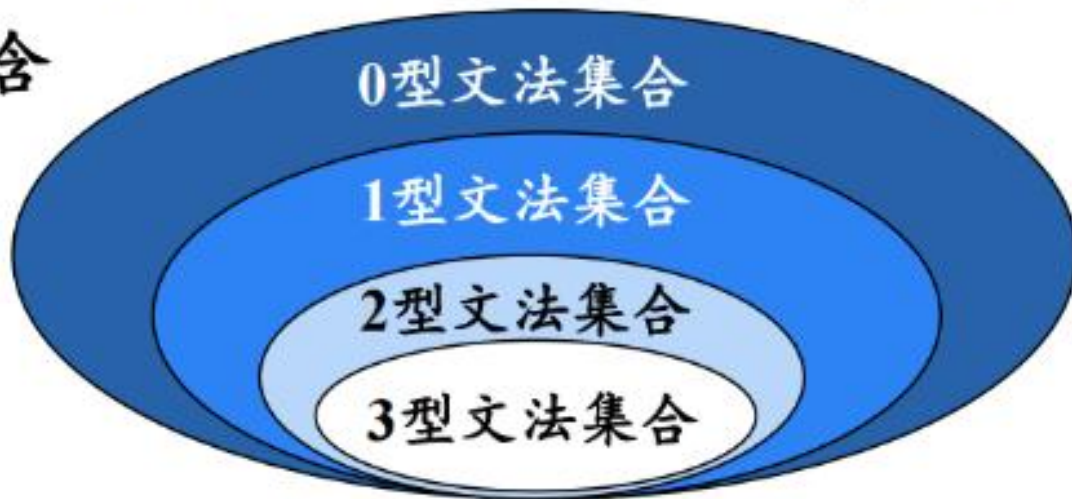
➤ 0型文法： α 中至少包含1个非终结符

➤ 1型文法 (CSG) : $|\alpha| \leq |\beta|$

➤ 2型文法 (CFG) : $\alpha \in V_N$

➤ 3型文法 (RG) : $A \rightarrow wB$ 或 $A \rightarrow w$ ($A \rightarrow Bw$ 或 $A \rightarrow w$)

➤ 逐级包含



由四类文法的定义可知，从0型文法到3型文法逐渐增加限制。1~3型文法都属于0型文法，2、3型文法均属于1型文法，3型文法属于2型文法。四类文法的区别如下：

- (1) 1型文法中不允许有形如“ $A \rightarrow \varepsilon$ ”的产生式存在，而2、3型文法则允许形如“ $A \rightarrow \varepsilon$ ”的产生式存在；
- (2) 0、1型文法的产生式左部存在含有终结符号的符号串或两个以上的非终结符，而2型和3型文法的产生式左部只允许是单个的非终结符号。

例3.3试判断下列产生式集所对应的文法和产生的语言：

- | | | | | | | | |
|-----|--------------------------------|-----|------------------------|-----|-----------------------|-----|----------------------|
| (1) | ① $S \rightarrow ACaB$ | (2) | ① $S \rightarrow aSBC$ | (3) | ① $S \rightarrow Ac$ | (4) | ① $S \rightarrow aS$ |
| | ② $Ca \rightarrow aaC$ | | ② $S \rightarrow aBC$ | | ② $S \rightarrow Sc$ | ② | $S \rightarrow aA$ |
| | ③ $CB \rightarrow DB$ | | ③ $CB \rightarrow DB$ | | ③ $A \rightarrow ab$ | ③ | $A \rightarrow bA$ |
| | ④ $CB \rightarrow E$ | | ④ $DB \rightarrow DC$ | | ④ $A \rightarrow aAb$ | ④ | $A \rightarrow bB$ |
| | ⑤ $aD \rightarrow Da$ | | ⑤ $DC \rightarrow BC$ | | ⑤ $B \rightarrow cB$ | | |
| | ⑥ $AD \rightarrow AC$ | | ⑥ $aB \rightarrow ab$ | ⑥ | $B \rightarrow c$ | | |
| | ⑦ $aE \rightarrow Ea$ | | ⑦ $bB \rightarrow bb$ | | | | |
| | ⑧ $AE \rightarrow \varepsilon$ | | | ⑧ | $bC \rightarrow bc$ | | |
| | ⑨ $cC \rightarrow cc$ | | | | | | |

[解答] 由四类文法的定义与区别可知, 1~4分别为0~3型文法。

(1) 该0型文法产生的0型语言为 $L_0(G) = \{a^{2n} \mid n > 0\}$ 。

例如: 当 $n=2$ 时, 句子 $a^{2 \times 2} = aaaa$,

$$\begin{aligned}
 S &\xRightarrow{(1)} ACaB \xRightarrow{(2)} AaaCB \xRightarrow{(3)} AaaDB \xRightarrow{(5)} AaDaB \xRightarrow{(6)} ADaaB \xRightarrow{(2)} ACaaB \xRightarrow{(2)} AaaCaB \xRightarrow{(2)} AaaaaCB \\
 &\xRightarrow{(1)} AaaaaE \xRightarrow{(4)} AaaaEa \xRightarrow{(7)} AaaEaa \xRightarrow{(7)} AaEaaa \xRightarrow{(7)} AEaaaa \xRightarrow{(8)} aaaa
 \end{aligned}$$

(2) 该1型文法产生的1型语言为

$L_1(G) = \{a^n b^n c^n \mid n \geq 1\}$ 。例如，当 $n=2$ 时，句子 $a^2 b^2 c^2 = aabbcc$ 是通过下列推导得到的：

$$\begin{aligned} S &\xRightarrow{(1)} aSBC \xRightarrow{(2)} aaBCBC \xRightarrow{(6)} aabCBC \xRightarrow{(3)} aabDBC \xRightarrow{(4)} aabDCC \xRightarrow{(5)} aabBCC \xRightarrow{(7)} aabbCC \xRightarrow{(8)} \\ &aabbccC \xRightarrow{(9)} aabbcc \end{aligned}$$

(3) 该2型文法产生的2型语言为 $L_2(G)=\{a^n b^n c^m \mid m, n \geq 1\}$ 。例如当 $n=2$ 、 $m=3$ 时，句子 $a^2 b^2 c^3=aabbccc$ 是通过下列推导得到的：

$$S \xRightarrow{(2)} Sc \xRightarrow{(2)} Scc \xRightarrow{(1)} Accc \xRightarrow{(4)} aAbccc \xRightarrow{(3)} aabbccc$$

(4) 该3型文法产生的3型语言为 $L_3(G)=\{a^m b^n c^k \mid m, n, k \geq 1\}$ 。例如当 $m=2$ 、 $n=3$ 、 $k=4$ 时，句子 $a^2 b^3 c^4 = aabb bccccc$ 是通过下列推导得到的：

$$\begin{array}{l} S \xRightarrow{(1)} aS \xRightarrow{(2)} aaA \xRightarrow{(3)} aabA \xRightarrow{(3)} aabbA \xRightarrow{(4)} aabbbB \xRightarrow{(5)} aabbbcB \xRightarrow{(5)} aabbbccB \xRightarrow{(5)} \\ aabbbcccB \xRightarrow{(6)} aabbbcccc \end{array}$$

由例3.3可知： $\{a^n b^n c^n \mid n \geq 1\}$ $\{a^n b^n c^m \mid m, n \geq 1\} \subset \{a^m b^n c^k \mid m, n, k \geq 1\}$ ，这说明对文法规则定义形式的限制虽然加强了，但相应的语言反而更大了。因此，不能主观认定文法限制越大则语言越小，也即下述结论是不成立的：

3型语言 $\subset$ 2型语言 $\subset$ 1型语言 $\subset$ 0型语言

在编译方法中，通常用3型文法来描述高级程序语言的词法部分，然后用有限自动机FA来识别高级语言的单词；利用2型文法来描述高级语言的语法部分，然后用下推自动机PDA来识别高级语言的各种语法成分。

3.1.3 正规表达式与上下文无关文法

1. 正规表达式到上下文无关文法的转换

正规表达式所描述的语言结构均可以用上下文无关文法描述，反之则不一定。由正规表达式构造上下文无关文法的一种方法如下：

- (1) 构造正规表达式的NFA；
- (2) 若0为初始状态，则 A_0 为开始符号；
- (3) 如果存在映射关系 $f(i, a) = j$ ，则定义产生式 $A_i \rightarrow aA_j$ ；
- (4) 如果存在映射关系 $f(i, \varepsilon) = j$ ，则定义产生式 $A_i \rightarrow A_j$ ；
- (5) 若 i 为终态，则定义产生式 $A_i \rightarrow \varepsilon$ 。

例3.5 用上下文无关文法描述正规表达式 $(a \mid b)^*abb$ 。

[解答] 首先构造识别正规表达式 $(a \mid b)^*abb$ 的NFA M 如图3-3所示。

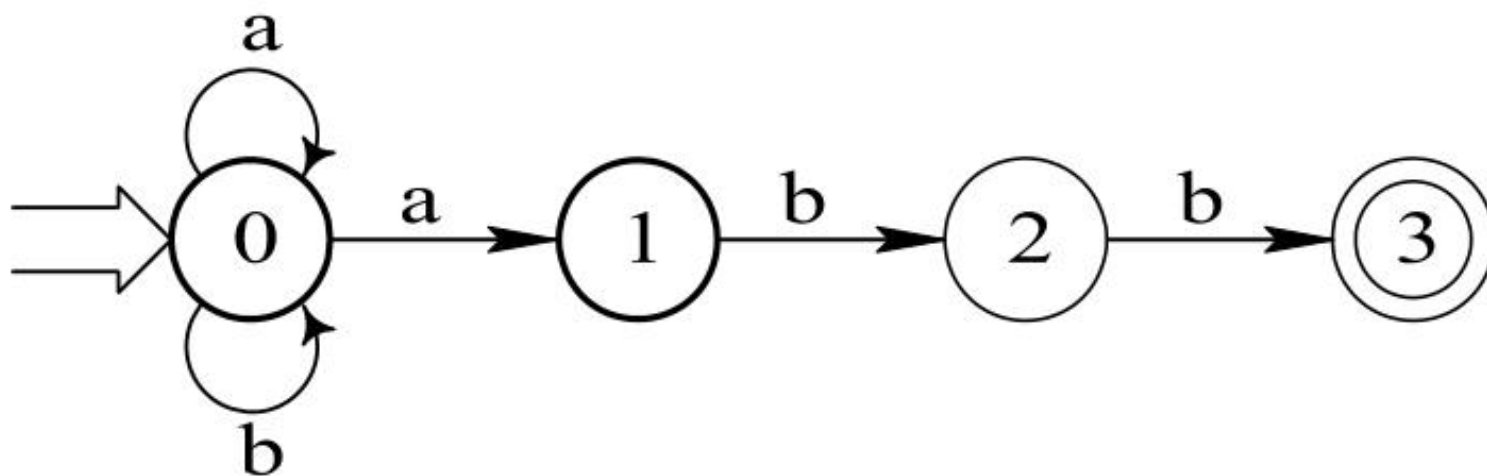


图3-3 例3.5的NFA M

例3.5 用上下文无关文法描述正规表达式 $(a \mid b)^*abb$ 。

[解答] 首先构造识别正规表达式 $(a \mid b)^*abb$ 的NFA M 如图3-3所示。

由构造上下文无关文法的方法得到上下文无关文法 $G[A_0]$ 如下：

$$G[A_0]: A_0 \rightarrow aA_0 \mid bA_0 \mid aA_1$$

$$A_1 \rightarrow bA_2$$

$$A_2 \rightarrow bA_3$$

$$A_3 \rightarrow \varepsilon$$

事实上，由正规表达式构造上下文无关文法还可以采用另一种方法，即通过分析正规表达式的特性凭经验直接构造。如可把 $(a \mid b)^*abb$ 看作由 $(a \mid b)^*$ 和 abb 两部分组成，第一部分是由0个或若干个 a 和 b 组成的字符串，而第二部分则仅由 abb 字符串组成，由此得到上下文无关文法 $G[A_0]$ 如下：

$$G[A_0]: A \rightarrow HT$$

$$H \rightarrow aH \mid bH \mid \varepsilon$$

$$T \rightarrow abb$$

3型文法产生的语言是有穷自动机（**FA**）所接受的集合.

定理 设 $G = (V_N, V_T, P, S)$ 是3型文法, 则存在一个有穷自动机 $M = (K, \Sigma, f, A, Z)$, 使得 $L(M) = L(G)$

有穷自动机NFA M 这样构造:

- $\Sigma = V_T$
- $K = V_N \cup \{N\}$, N 为一个新状态, 它不在 V_N 中
- $A = S$
- $Z = \{N\}$
- 对 G 中的形如 $D \rightarrow tB$ 的产生式, t 为终结符或 ϵ , 有 $f(D, t) = B$;
对 G 中形如 $D \rightarrow t$ 的产生式, t 为终结符或 ϵ , 有 $f(D, t) = N$;
对 V_T 中的每一个 a , 有 $f(N, a) = \phi$

3型文法 和有穷自动机 (FA)

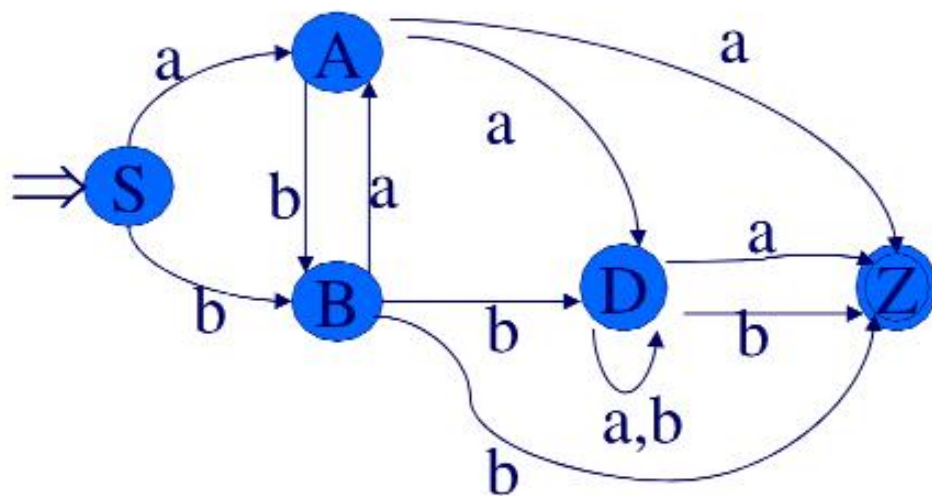
$G[S]$:

$S \rightarrow aA | bB$

$A \rightarrow bB | aD | a$

$B \rightarrow aA | bD | b$

$D \rightarrow aD | bD | a | b$

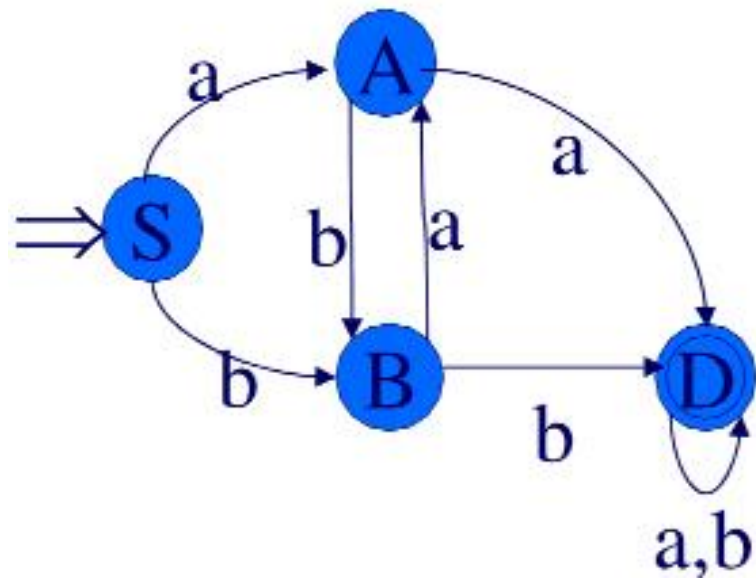


3型文法 和有穷自动机 (FA)

定理 已知一有穷自动机 $M = (K, \Sigma, f, A, Z)$,
存在有一个3型文法 $G = (V_N, V_T, P, S)$, 使得 $L(G) = L(M)$

G 的定义:

- $V_T = \Sigma$ • $V_N = K$
- $S = A$
- 若 $f(D, t) = B$, 则 $D \rightarrow tB$ 在 P 中
若 $f(D, t) = B$, 且 B 在 Z 中, 则 $D \rightarrow t$ 在 P 中



$G [S] :$

$S \rightarrow aA | bB$

$A \rightarrow bB | aD | a$

$B \rightarrow aA | bD | b$

$D \rightarrow aD | bD | a | b$

程序设计语言的语法通常由上下文无关的文法规则给出。它利用了与正规表达式中极为类似的命名惯例和运算，但二者间有一个主要区别，即：上下文无关文法的规则是递归的。例如：嵌套if语句。因此，必须使用栈式数据结构来表示语法结构。

$$r=1(1 \mid d)^*$$

〈标识符〉→ 〈字母〉

〈标识符〉→ 〈标识符〉〈字母〉

〈标识符〉→ 〈标识符〉〈数字〉

〈字母〉 → a, ..., 〈字母〉→ z

〈数字〉 → 0, ..., 〈数字〉→ 9

文法实用中的一些说明

- 文法中不应含有有害规则和多余规则

有害规则：形如 $U \rightarrow U$ 的产生式。会引起文法的二义性

多余规则：指文法中对任何句子的推导都不会用到的规则

- 文法中不含有不可到达和不可终止的非终结符

不可到达非终结符：文法中某些非终结符不在任何规则的右部出现

不可终止非终结符：文法中某些非终结符，由它不能推出终结符号串

对于文法 $G[S]$ ，为了保证任一非终结符 A 在某个句子推导中出现，必须满足如下两个条件：

➤ A 必须在某句型中出现

即有 $S \xRightarrow{*} \alpha A \beta$ ，其中 α, β 属于 V^*

➤ 必须能够从 A 推出终结符号串 t 来

即 $A \xRightarrow{*} t$ ，其中 $t \in V_T^*$

例： $G[S]$ ：

1) $S \rightarrow Be$

2) $B \rightarrow Ce$

3) $B \rightarrow Af$

4) $A \rightarrow Ae$

5) $A \rightarrow e$

6) $C \rightarrow Cf$

7) $D \rightarrow f$

C 为不可终止

D 为不可到达

产生式 2)，6)，7)
为多余规则应去掉。

上下文无关文法中的 ϵ 规则

上下文无关文法中某些规则可具有形式 $A \rightarrow \epsilon$, 称这种规则为 ϵ 规则。因为 ϵ 规则会使得有关文法的一些讨论和证明变得复杂, 有时会限制这种规则的出现。

文法构思的启示是要找出**语言的有穷描述**, 而如果语言 L 有一个有穷的描述, 则 $L_1 = L \cup \{\epsilon\}$ 也同样有一个有穷的描述, 并且可以证明, **若 L 是上下文有关语言、上下文无关语言或正规语言, 则 $L \cup \{\epsilon\}$ 和 $L - \{\epsilon\}$ 同样分别是上下文有关语言、上下文无关语言或正规语言。**

1. 语法树

对程序语言来说，有两个问题需要解决：其一是判别程序在语法上是否正确；其二是句子的识别或分析。在编译方法中，为了便于识别或分析句子而引入了语法树这一重要的辅助工具。语法树以图示化的形式把句子分解成各个组成部分来描述或分析句子的语法结构，这种图示化的表示与所定义的文法规则完全一致，但更为直观和完整。

4. 4、上下文无关文法及其语法树

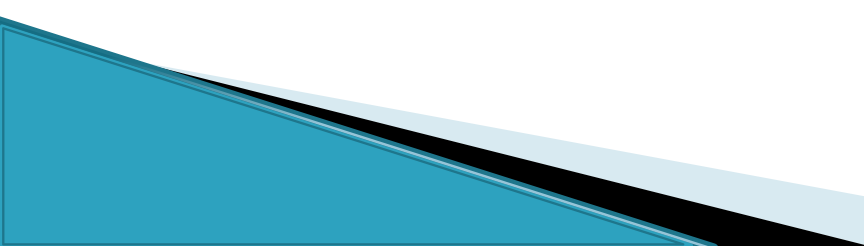
设 $G=(V_N, V_T, P, S)$ ，若**一棵树**满足下列4个条件，则此树称作 G 的**语法树**：

1. **每个结点都有一个标记**，此标记是 V 的一个符号
 2. 根的标记是 S
 3. 若一标记为 A 的结点至少有一个除它自己外的子孙，则肯定 $A \in V_N$ ；若结点无子孙，则称其为叶子，叶子一定是终结符。
 4. 如果标记为 A 的结点其**直接子孙结点**从左到右的次序是 $n_1, n_2, \dots, n_k$ ，其标记分别为 $A_1, A_2, \dots, A_k$ ，那么 **$A \rightarrow A_1 A_2 \dots A_k$ 一定是 P 中的一个产生式**
- 从左到右读出叶子的标记而构成的串谓之句子**

对文法 $G=(V_T, V_N, S, \xi)$ ，满足下列条件的树称为 $G[S]$ 的语法树：

- (1) 每个结点用 $G[S]$ 的一个终结符或非终结符标记；
- (2) 根结点用文法开始符 S 标记；
- (3) 内部结点(指非树叶结点)一定是非终结符，如果某内部结点 A 有 n 个分支，它的所有子结点从左至右依次标记为 x_1 、 x_2 、 $\dots$ 、 x_n ，则 $A \rightarrow x_1x_2\dots x_n$ 一定是文法 $G[S]$ 的一条产生式；
- (4) 如果某结点标记为 ε ，则它必为叶结点且是其父结点的惟一子结点。

相应于一个句型的语法树是以文法的开始符S作为根结点的，并随着推导逐步展开；当某个非终结符被它对应的产生式右部的某个候选式所替换时，这个非终结符所对应的结点就产生出下一代新结点，即候选式中从左至右的每一个符号都依次顺序对应一个新结点，且每个新结点与其父结点之间都有一条连线(树枝)。在一棵语法树生长过程中的任何时刻，所有那些没有后代的树叶结点自左至右排列起来就是一个句型。



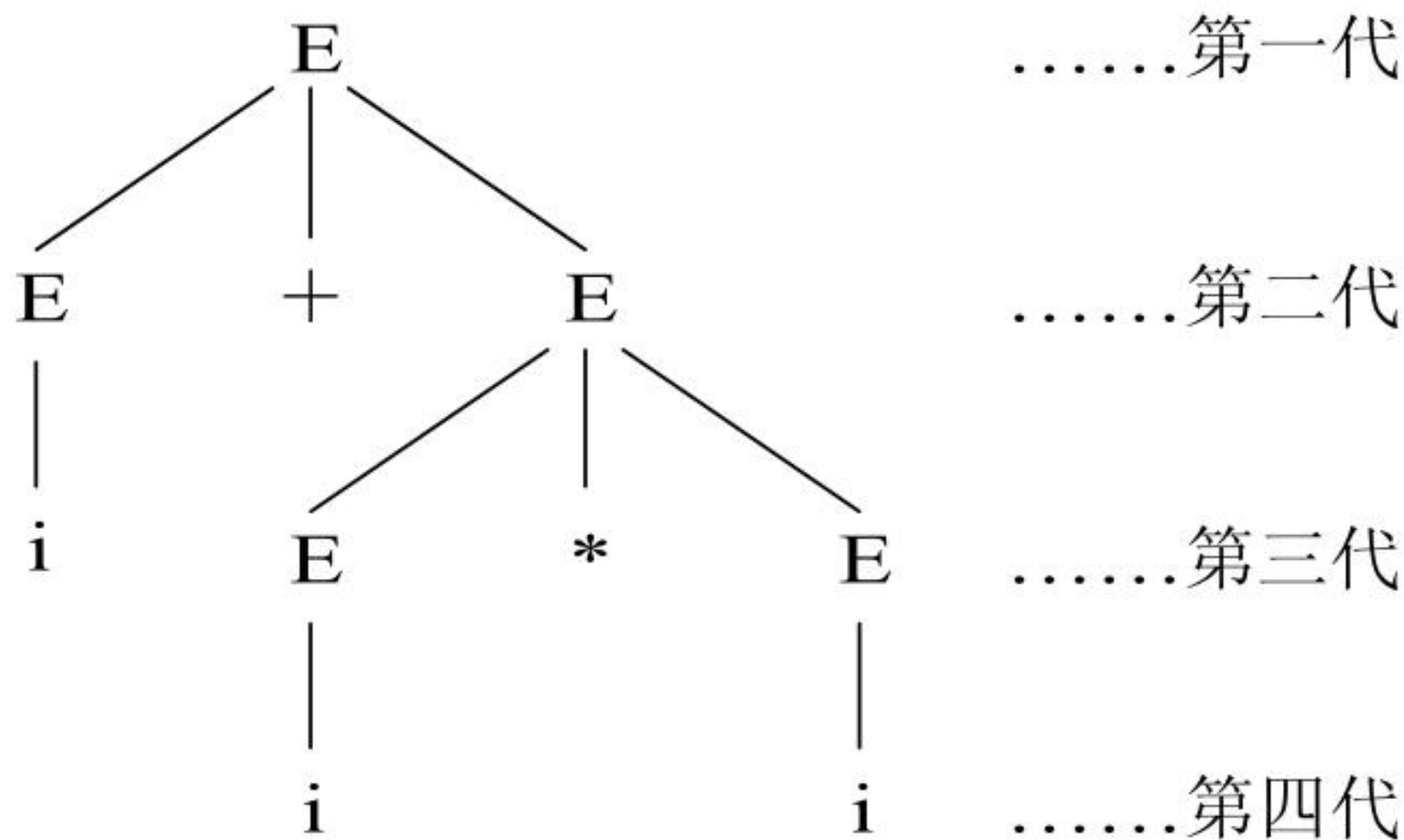


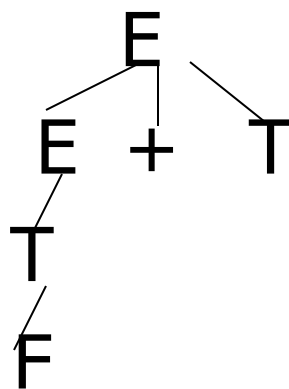
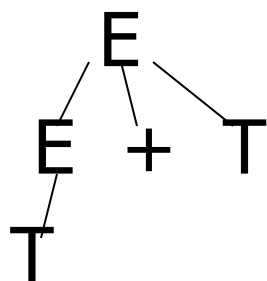
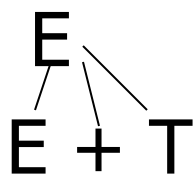
图3-4 句子 $i+i*i$ 的语法树

语法树—句型推导的直观表示

给定文法 $G=(V_N, V_T, P, S)$ ，对于 G 的任何句型都能构造与之关联的语法子树

$G[E]:$
$$\begin{array}{l|l} E \rightarrow E+T & T \\ T \rightarrow T * F & F \\ F \rightarrow (E) & a \end{array}$$

1) $E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T$
 $\Rightarrow a+T * F \Rightarrow a+F * F \Rightarrow a+a * F \Rightarrow a+a * a$



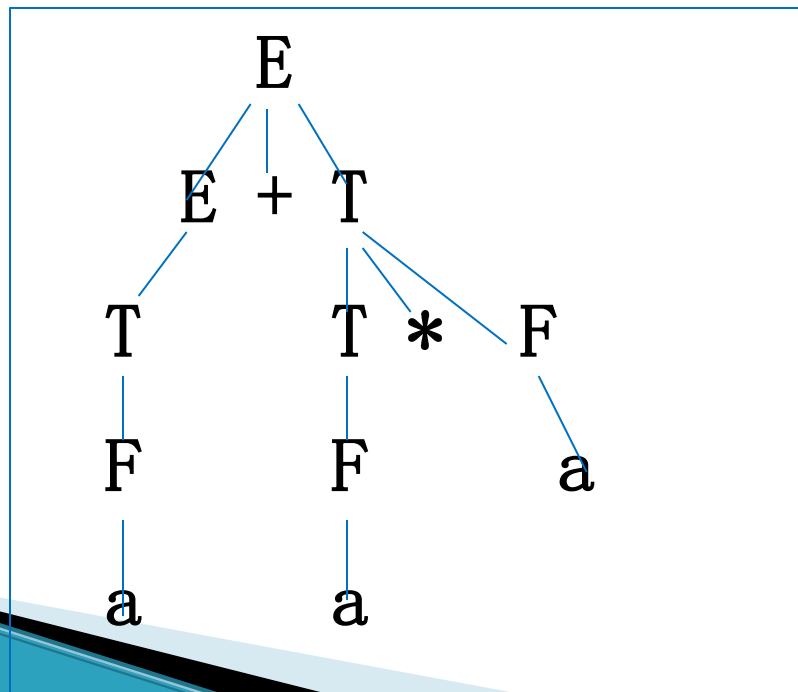
.....

2) $E \Rightarrow E+T \Rightarrow E+T^*F \Rightarrow E+T^*a \Rightarrow E+F^*a$

$\Rightarrow E+a^*a \Rightarrow T+a^*a \Rightarrow F+a^*a \Rightarrow a+a^*a$

3) $E \Rightarrow E+T \Rightarrow T+T \Rightarrow T+T^*F \Rightarrow F+T^*F$

$\Rightarrow F+F^*F \Rightarrow a+F^*F \Rightarrow a+F^*a \Rightarrow a+a^*a$



看不出句型的推导顺序，即这棵语法树可以表示这三种推导。

规范推导 规范句型

最左（最右）推导：在推导的任何一步

$\alpha \Rightarrow \beta$ ，其中 α 、 β 是句型，都是对 α 中的**最左（右）非终结符**进行替换。上面例子中的1）是最左推导，2）是最右推导。

最右推导被称为**规范推导**。由规范推导所得的句型称为**规范句型**（例如：T+T 就不是规范句型。）

$$E \Rightarrow E+E \Rightarrow i+E \Rightarrow i+E * E \Rightarrow i+i * E \Rightarrow i+i * i$$

从一个句型到另一个句型的推导过程并不是惟一的，为了对句子的结构进行确定性分析，我们往往只考虑最左推导或最右推导，并且称最右推导为规范推导。规范推导的逆过程便是规范归约。

在构造语法树时可以发现，一个句型的最左推导及最右推导只是决定先生长左子树还是先生长右子树，句型推导结束时相应的语法树也随之完成，这时已不能看出是先生长左子树还是先生长右子树，所呈现的只是已经长成的这个句子或句型的语法树，这与使用文法规则进行推导是有差异的，即使用文法规则的推导过程是有先后之分的。

因此，一棵语法树表示了一个句型种种可能的(但未必是所有的——见下面文法的二义性)不同推导过程，包括最左(最右)推导。如果我们坚持使用最左(或最右)推导，那么一棵语法树就完全等价于一个最左(或最右)推导，这种等价性也包括语法树的每一步生长和推导的每一步展开的这种完全一致性。

练习：对所给文法： $S \rightarrow (L)|a$ $L \rightarrow L,S|S$

(1) 指出文法的开始符号、终结符、非终结符；

(2) 为下述句子建立最左推导和最右推导，并给出它们最终的语法树；

① (a,a)

② $(a,(a,a))$

(3) 用自然语言描述该文法所产生的语言。

一棵语法树表示了一个句型的种种可能的(但未必是所有的)不同推导过程, 包括最左(最右)推导。但是:

- 一个句型是否只对应唯一的一棵语法树呢?
- 一个句型是否只有唯一的一个最左(最右)推导呢?

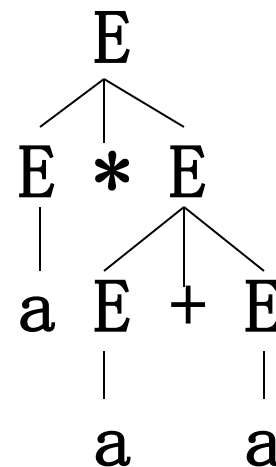
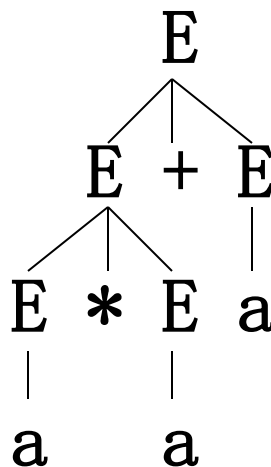
例: $G[E]$:

1 $E \rightarrow a$

2 $E \rightarrow E + E$

3 $E \rightarrow E * E$

4 $E \rightarrow (E)$



句型(句子) $a*a+a$ 的两个不同的最左推导:

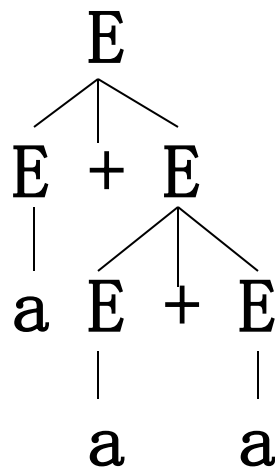
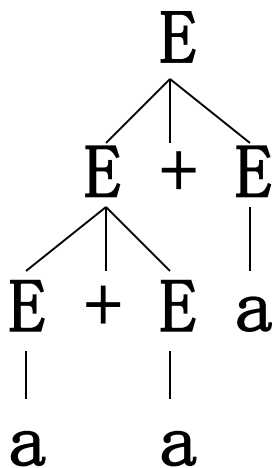
推导1: $\underline{E} \Rightarrow \underline{E} + E \Rightarrow \underline{E} * E + E \Rightarrow a * \underline{E} + E \Rightarrow a * a + \underline{E} \Rightarrow a * a + a$

推导2: $\underline{E} \Rightarrow \underline{E} * E \Rightarrow a * \underline{E} \Rightarrow a * \underline{E} + E \Rightarrow a * a + \underline{E} \Rightarrow a * a + a$

句型(句子) **a+a+a** 的两个不同的最左推导:

推导1: $\underline{E} \Rightarrow \underline{E}+E \Rightarrow \underline{E}+E+E \Rightarrow a+\underline{E}+E \Rightarrow a+a+\underline{E} \Rightarrow a+a+a$

推导2: $\underline{E} \Rightarrow \underline{E}+E \Rightarrow a+\underline{E} \Rightarrow a+\underline{E}+E \Rightarrow a+a+\underline{E} \Rightarrow a+a+a$



二义文法

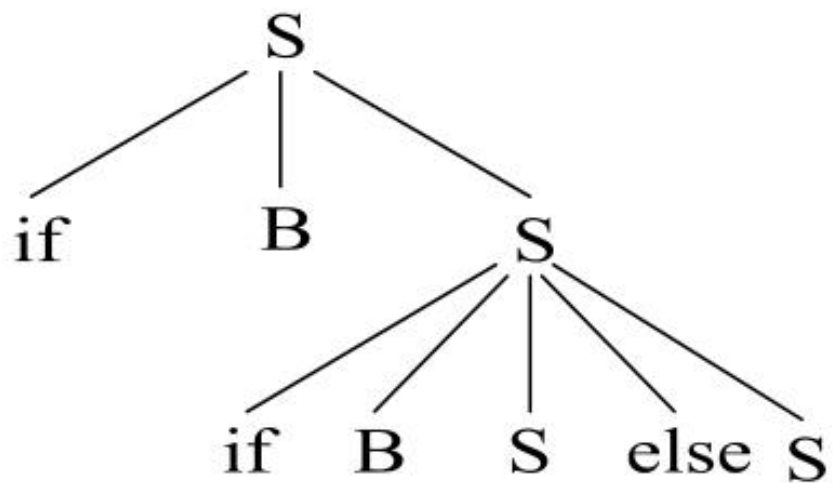
- 若一个文法存在**某个句子**有两个不同的最左（右）推导，则称这个文法是**二义**的。
- 某种意义上，二义性文法就像是一个**非确定的自动机**，此时**两个不同的路径都可接收相同的串**。
- 判定任给的一个上下文无关文法是否二义是递归不可解的，这和可以用子集法将NFA确定化不同。
- **文法的二义性和语言的二义性**是两个不同的概念。因为可能有两个不同的文法 G 和 G' ，其中 G 是二义的，而 G' 是非二义的，但是却有 $L(G)=L(G')$ ，也就是说，这两个文法所产生的语言是相同的。例如：

$$\begin{array}{l} G[E]: E \rightarrow E+T \mid T \\ \quad T \rightarrow T * F \mid F \\ \quad F \rightarrow (E) \mid a \end{array}$$
$$\begin{array}{l} G[E]: E \rightarrow a \\ \quad E \rightarrow E+E \\ \quad E \rightarrow E * E \\ \quad E \rightarrow (E) \end{array}$$

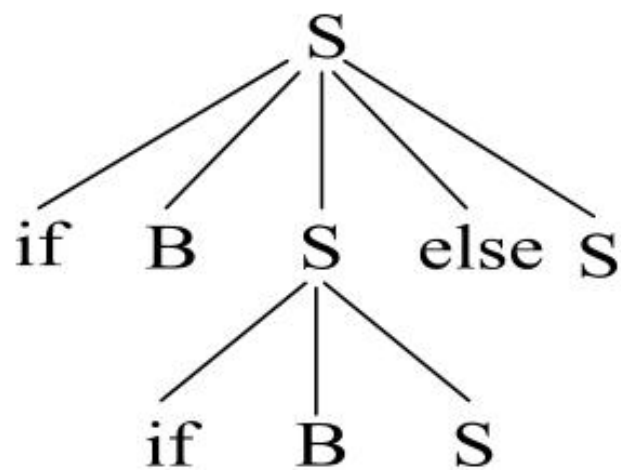
再如，条件语句的文法 $G[S]$ 为：

$$G[S]: S \rightarrow \text{if } B \ S$$
$$S \rightarrow \text{if } B \ S \ \text{else } S$$
$$S \rightarrow A \quad \quad \quad /*A \text{指其它语句}*/$$

其中， $V_N = \{B, S, A\}$ ， $V_T = \{\text{if}, \text{else}\}$ ，则句型if B
if B S else S所对应的两棵不同语法树见图3-7。因此，
文法 $G[S]$ 是二义性文法。



(a)



(b)

图3-7 句型 if B if B S else S 的两棵不同语法树

将二义语法改造为无二义语法

但有两个解决二义性的基本方法。

- 一是**消除二义性规则**：该规则可在每个二义性的情况下指出哪一个语法树正确。例如：规定*比+优先级高；+、*都为左结合的。它的**优点**是无需修改文法，**缺点**在于不能由文法单纯地决定语法结构。
- 二是**改造二义语法为非二义语法**。

将二义文法改造为无二义文法

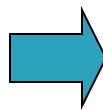
- 二是改造二义文法为非二义文法。

$G[E]: E \rightarrow a$

$E \rightarrow E+E$

$E \rightarrow E * E$

$E \rightarrow (E)$



$G[E]: E \rightarrow T \mid E+T$

$T \rightarrow F \mid T * F$

$F \rightarrow (E) \mid a$

新的文法对 $a * a + a$ 、 $a + a + a$ 只有一个最左（右）推导，因此是非二义文法。

悬挂else问题

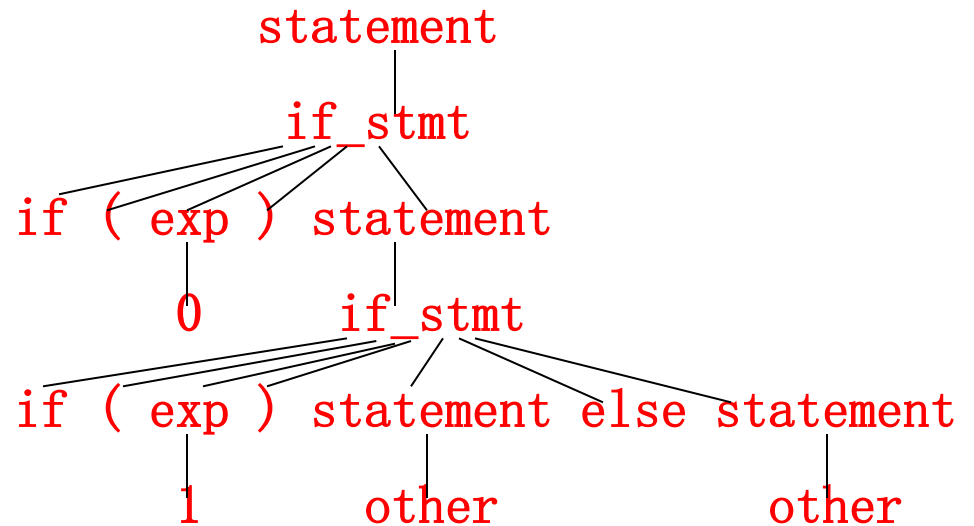
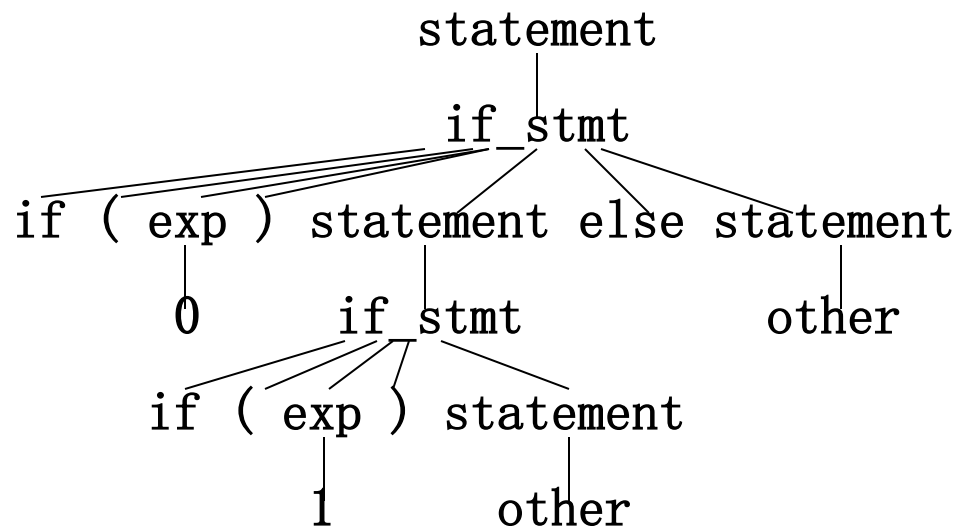
考虑下面文法：

$$\begin{aligned}\langle \text{statement} \rangle &\rightarrow \langle \text{if_stmt} \rangle \mid \langle \text{other} \rangle \\ \langle \text{if_stmt} \rangle &\rightarrow \text{if}(\langle \text{exp} \rangle) \langle \text{statement} \rangle \\ &\quad \mid \text{if}(\langle \text{exp} \rangle) \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle \\ \langle \text{exp} \rangle &\rightarrow 0 \mid 1\end{aligned}$$

这个文法具有二义性。例如串：

`if(0) if(1) <other> else <other>`

if(0) if(1) <other> else <other>



用前面所说的两种方法解决悬挂else问题的方法

- 消除二义性规则：最近嵌套
- 改写文法：

$\langle \text{statement} \rangle \rightarrow \langle \text{if_stmt} \rangle \mid \langle \text{otherstmt} \rangle$

$\langle \text{if_stmt} \rangle \rightarrow \text{if}(\langle \text{exp} \rangle) \text{ then } \langle \text{statement} \rangle \text{ end if}$
 $\quad \mid \text{if}(\langle \text{exp} \rangle) \text{ then } \langle \text{statement} \rangle \text{ else } \langle \text{statement} \rangle \text{ end if}$

$\langle \text{exp} \rangle \rightarrow 0 \mid 1$

if(0) if(1) $\langle \text{other} \rangle$ else $\langle \text{other} \rangle$ 变换为：

```
if(0) then
    if(1) then  $\langle \text{other} \rangle$ 
    end if
else  $\langle \text{other} \rangle$ 
end if
```

```
if(0) then
    if(1) then  $\langle \text{other} \rangle$ 
        else  $\langle \text{other} \rangle$ 
    end if
end if
```

我们总希望一个文法是无二义性的，这样，句子的分析可以按惟一确定的方式进行。但是，文法的二义性是不可判定的，即不存在一种算法，能够在有限步内判定一个文法是否为二义性的。有时候，二义性文法也可带来一定的好处，如语法分析中二义性文法的应用。



练习1：对所给文法G： $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

- (1) 通过为某个该文法所产生的句子建立两个最左推导来说明G是二义文法，给出两个最左推导的语法树；
- (2) 另外举出两个G产生的句子并用自然语言描述该文法所产生的语言。

作业：假设有文法 $A \rightarrow AA \mid (A) \mid \varepsilon$

- a. 描述它生成的语言
- b. 说明它有二义性

选做：下述条件语句文法G试图消除悬空else，试证明G仍然是二义的。

$$\begin{aligned} \langle \text{statement} \rangle \rightarrow & \text{if}(\langle \text{expr} \rangle) \text{ then } \langle \text{statement} \rangle \\ & \mid \langle \text{matched_stmt} \rangle \end{aligned}$$
$$\begin{aligned} \langle \text{matched_stmt} \rangle \rightarrow & \text{if}(\langle \text{expr} \rangle) \text{ then } \langle \text{matched_stmt} \rangle \text{ else } \langle \text{statement} \rangle \\ & \mid \langle \text{statement} \rangle \end{aligned}$$
$$\langle \text{exp} \rangle \rightarrow 0 \mid 1$$